Translating Modules

This section explains how to provide translation abilities to your module.

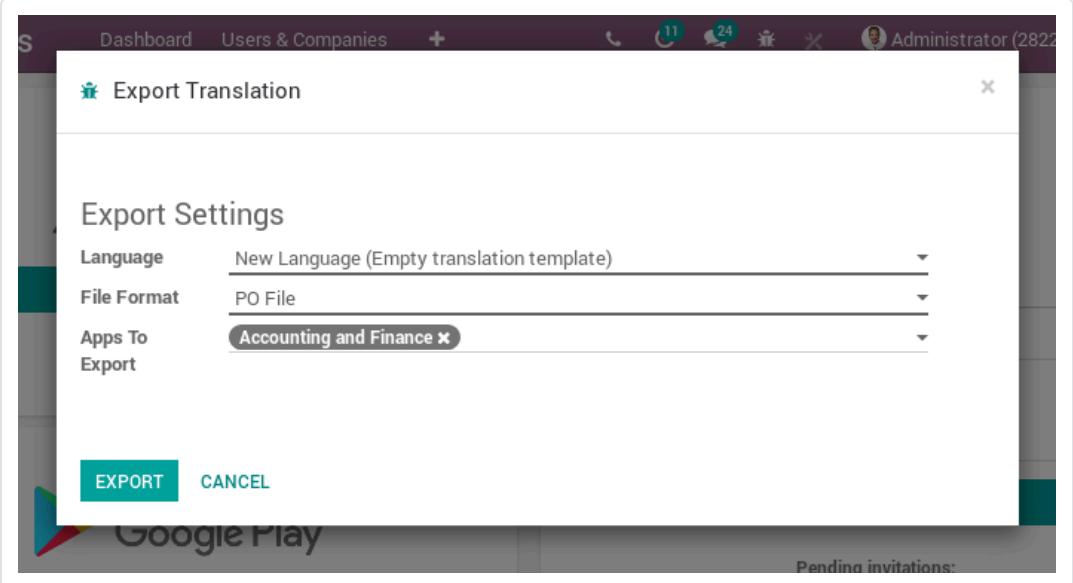
> **Note**
>
> If you want to contribute to the translation of Odoo itself, please refer to the **Odoo Wiki page**.

Exporting translatable term

A number of terms in your modules are implicitly translatable. As a result, even if you haven't done any specific work towards translation, you can export your module's translatable terms and may find content to work with.

Translations export is performed via the administration interface by logging into the backend interface and opening **Settings** ▸ **Translations** ▸ **Import / Export** ▸ **Export Translations**

- leave the language to the default (new language/empty template)
- select the **PO File** format
- select your module
- click **Export** and download the file



dedicated translation tool like POEdit or by simply copying the template to a new file called *language.po*. Translation files should be put in *yourmodule/i18n/*, next to *yourmodule.pot*, and will be automatically loaded by Odoo when the corresponding language is installed (via **Settings ▸ Translations ▸ Languages**)

> **Note**
>
> translations for all loaded languages are also installed or updated when installing or updating a module

Implicit exports

Odoo automatically exports translatable strings from "data"-type content:

- in non-QWeb views, all text nodes are exported as well as the content of the `string`, `help`, `sum`, `confirm` and `placeholder` attributes
- QWeb templates (both server-side and client-side), all text nodes are exported except inside `t-translation="off"` blocks, the content of the `title`, `alt`, `label` and `placeholder` attributes are also exported
- for **`Field`**, unless their model is marked with `_translate = False`:
  - their `string` and `help` attributes are exported
  - if `selection` is present and a list (or tuple), it's exported
  - if their `translate` attribute is set to `True`, all of their existing values (across all records) are exported

Explicit exports

When it comes to more "imperative" situations in Python code or Javascript code, Odoo cannot automatically export translatable terms so they must be marked explicitly for export. This is done by wrapping a literal string in a function call.

In Python, the wrapping function is **`odoo.api.Environment._()`** and **`odoo.tools.translate._()`**:

```
title = self.env._("Bank Accounts")

# old API for backward-compatibility
```

In JavaScript, the wrapping function is generally **`odoo.web._t()`** :

```
title = _t("Bank Accounts");
```

> **Warning**
>
> Only literal strings can be marked for exports, not expressions or variables. For situations where strings are formatted, this means the format string must be marked, not the formatted string

The lazy version of `_` and `_t` is the **`odoo.tools.translate.LazyTranslate`** factory in python and **`odoo.web._lt()`** in javascript. The translation lookup is executed only at rendering and can be used to declare translatable properties in class methods of global variables.

```
from odoo.tools import LazyTranslate
_lt = LazyTranslate(__name__)
LAZY_TEXT = _lt("some text")
```

> **Note**
>
> Translations of a module are **not** exposed to the front end by default and thus are not accessible from JavaScript. In order to achieve that, the module name has to be either prefixed with `website` (just like `website_sale`, `website_event` etc.) or explicitly register by implementing **`_get_translation_frontend_modules_name()`** for the `ir.http` model.
>
> This could look like the following:

```
from odoo import models

class IrHttp(models.AbstractModel):
    _inherit = ['ir.http']

    @classmethod
    def _get_translation_frontend_modules_name(cls):
```

odoo docs

Context

To translate, the translation function needs to know the *language* and the *module* name. When using `Environment._` the language is known and you may pass the module name as a parameter, otherwise it's extracted from the caller.

In case of `odoo.tools.translate._`, the language and the module are extracted from the context. For this, we inspect the caller's local variables. The drawback of this method is that it is error-prone: we try to find the context variable or `self.env`, however these may not exist if you use translations outside of model methods; i.e. it does not work inside regular functions or python comprehensions.

Lazy translations are bound to the module during their creation and the language is resolved when evaluating using `str()`. Note that you can also pass a lazy translation to `Environment._` to translate it without any magic language resolution.

Variables

Don't the extract may work but it will not translate the text correctly:

```
_("Scheduled meeting with %s" % invitee.name)
```

Do set the dynamic variables as a parameter of the translation lookup (this will fallback on source in case of missing placeholder in the translation):

```
_("Scheduled meeting with %s", invitee.name)
```

Blocks

Don't split your translation in several blocks or multiples lines:

```
# bad, trailing spaces, blocks out of context
_("You have ") + len(invoices) + _(" invoices waiting")
_t("You have ") + invoices.length + _t(" invoices waiting");

# bad, multiple small translations
```

odoo docs

Do keep in one block, giving the full context to translators:

```
# good, allow to change position of the number in the translation
_("You have %s invoices waiting") % len(invoices)
_.str.sprintf(_t("You have %s invoices waiting"), invoices.length);

# good, full sentence is understandable
_("Reference of the document that generated " + \
  "this sales order request.")
```

Plural

Don't pluralize terms the English-way:

```
msg = _("You have %(count)s invoice", count=invoice_count)
if invoice_count > 1:
  msg += _("s")
```

Do keep in mind every language has different plural forms:

```
if invoice_count > 1:
  msg = _("You have %(count)s invoices", count=invoice_count)
else:
  msg = _("You have one invoice")
```

Read vs Run Time

Don't invoke translation lookup at server launch:

```
ERROR_MESSAGE = {
  # bad, evaluated at server launch with no user language
  'access_error': _('Access Error'),
  'missing_error': _('Missing Record'),
}

class Record(models.Model):
```

odoo docs

Don't invoke translation lookup when the javascript file is read:

```
# bad, js _t is evaluated too early
var core = require('web.core');
var _t = core._t;
var map_title = {
    access_error: _t('Access Error'),
    missing_error: _t('Missing Record'),
};
```

Do use lazy translation lookup method:

```
ERROR_MESSAGE = {
  'access_error': _lt('Access Error'),
  'missing_error': _lt('Missing Record'),
}

class Record(models.Model):

  def _raise_error(self, code):
    # translation lookup executed at error rendering
    raise UserError(ERROR_MESSAGE[code])
```

or **do** evaluate dynamically the translatable content:

```
# good, evaluated at run time
def _get_error_message(self):
  return {
    access_error: _('Access Error'),
    missing_error: _('Missing Record'),
  }
```

Do in the case where the translation lookup is done when the JS file is *read*, use `_lt` instead of `_t` to translate the term when it is *used*:

```
var map_title = {
    access_error: _lt('Access Error'),
    missing_error: _lt('Missing Record'),
};
```

Get Help

Contact Support | Ask the Odoo Community